INSTITUTO POLITÉCNICO NACIONAL

ESCUELA SUPERIOR DE CÓMPUTO

GRUPO 3CV4

GESTOR DE CITAS MÉDICAS

Entrega Parcial 1 · Etapa Funcional

Materia: Paradigmas de Programación
Profesora: Martínez Hernández Guadalupe Ana Gabriela

Integrantes

Contreras Cruz Joseph
Mendez Gamboa Santiago
Rosas Rios Luis Gabriel

AVISO DE ALCANCE · SEGUNDO PARCIAL

El presente documento conserva las 25 páginas del reporte del primer parcial y agrega el desarrollo del Producto Mínimo Viable para la segunda entrega de Paradigmas de Programación (11 de noviembre de 2026).

Interpretación del reporte anterior

Los requerimientos R01 a R14 incluidos en las primeras páginas describen la visión general del Gestor de Citas Médicas. No todos forman parte del software ya implementado. Para evitar interpretaciones equivocadas durante las tres pruebas aleatorias, la sección 8 delimita las funciones demostrables y la sección 12 distingue explícitamente las no desarrolladas.

Alcance que sí ofrece el MVP actual

- Inicio y cierre de sesión con cinco cuentas de demostración y control de permisos.
- Búsqueda de horarios y prevención de citas que se empalmen.
- Registro y cancelación de citas por recepción o administración.
- Consulta de agenda restringida al doctor o al paciente autenticado.
- Alta de médicos y vista de resumen para administración.
- Consultas sin acceso a datos personales para personal autorizado.

La implementación es un prototipo LOCAL con datos ficticios. El perfil “personal autorizado” se presenta como un acceso limitado para la demostración, aun cuando en el documento original también es una categoría general de usuarios.

Las funciones de reprogramación, administración completa de cuentas, notificaciones y ausencia de doctor siguen identificadas como ampliaciones futuras.

1. Recuperación del trabajo previo

Para esta primera entrega retomamos la idea que traíamos desde el avance anterior: un gestor de citas médicas. La intención es que el sistema pueda guardar y organizar la información de pacientes, doctores, horarios y las citas como tal. Con eso cubrimos las operaciones que normalmente se esperarían de una herramienta así: registrar, consultar, agendar, cancelar y reprogramar, además de mandar confirmaciones y cuidar los datos personales.

En cuanto a las tecnologías, no nos movimos mucho respecto a lo que ya habíamos planteado. Python se queda como lenguaje principal porque es multiparadigma y eso nos deja usar el enfoque que más nos convenga en cada parte. JavaScript lo usamos del lado del navegador, SQL queda para la persistencia y HTML/CSS para la estructura y presentación de la página.

1.1 Requerimientos

ID	Requerimiento	Actor
R01	Registrar pacientes en el sistema	Recepción
R02	Registrar doctores y sus especialidades	Administrador
R03	Permitir iniciar sesión a los usuarios	Usuarios autorizados
R04	Mostrar los horarios disponibles de los doctores	Recepción / Paciente
R05	Permitir agendar una cita médica	Recepción
R06	Impedir agendar una cita cuando el doctor ya tenga otra cita en ese horario	Recepción
R07	Permitir cancelar una cita	Recepción / Administrador
R08	Permitir reprogramar una cita	Recepción / Administrador
R09	Mostrar al paciente sus citas próximas	Paciente
R10	Permitir al doctor consultar su agenda	Doctor
R11	Permitir al administrador agregar, modificar o eliminar usuarios	Administrador
R12	Enviar una confirmación cuando se registre una cita	Automático
R13	Mantener protegidos los datos personales de pacientes y doctores	Sistema
R14	Permitir al doctor notificar que no asistirá para cancelar su lista	Doctor

1.2 Roles y actores

Actor	Responsabilidades principales

Actor	Responsabilidades principales
Administrador	Gestiona usuarios y operaciones administrativas.
Doctor	Consulta agenda y horarios; puede notificar que no asistirá.
Paciente	Consulta sus citas próximas y la información necesaria para su atención.
Recepción	Registra pacientes y realiza el agendado y gestión de citas.
Personal autorizado	Accede a funciones asignadas por el sistema.

1.3 Casos de uso

ID	Caso de uso	Actor principal
CU01	Registrarse como paciente	Recepción / Paciente
CU02	Registrarse como doctor	Administrador
CU03	Consultar horarios disponibles	Recepción / Paciente
CU04	Agendar cita	Recepción
CU05	Cancelar cita	Recepción / Administrador
CU06	Reprogramar cita	Recepción / Administrador
CU07	Consultar calendario de citas	Doctor / Recepción
CU08	Consultar doctores disponibles	Paciente / Recepción
CU09	Administrar usuarios	Administrador
CU10	Modificar datos del paciente	Recepción / Administrador
CU11	Envío de notificación al paciente y al doctor	Sistema

1.4 Elección y evaluación de lenguajes

Elegimos Python como lenguaje principal por varias razones. La primera es que permite mezclar sin problema programación imperativa, orientada a objetos y funcional dentro del mismo proyecto, que es justo lo que necesitábamos para esta etapa. A eso se suma que la sintaxis es bastante legible, el tipado es dinámico pero fuerte, no tenemos que preocuparnos por administrar memoria y podemos tratar las funciones como cualquier otro valor. Tampoco hay que olvidar que tiene un ecosistema muy amplio para aplicaciones web, bases de datos y pruebas, lo cual nos ahorra bastante trabajo.

JavaScript lo vamos a usar del lado del frontend: eventos, validaciones del formulario y actualizaciones dinámicas de la interfaz. SQL queda reservado para la persistencia relacional y HTML/CSS para la estructura y el diseño visual.

1.5 Distribución propuesta del sistema

Tecnología	Responsabilidad

Tecnología	Responsabilidad
Python	Lógica del servidor, reglas de negocio, autenticación y validación funcional.
JavaScript	Interacción del frontend, formularios, selección de horarios y consumo de la API.
SQL	Persistencia futura de usuarios, médicos, pacientes, horarios y citas.
HTML/CSS	Estructura y diseño visual de la aplicación web.

2. Selección de una funcionalidad

2.1 Funcionalidad seleccionada: búsqueda de horarios disponibles de un médico

De todo el sistema, la funcionalidad que decidimos desarrollar a fondo fue la búsqueda de horarios disponibles de un médico. La función recibe la agenda del médico, su horario laboral y la duración que se quiere para la cita, y con eso calcula los espacios libres donde esa cita podría colocarse sin empalmarse con otra. La elegimos porque era justo la que ya habíamos empezado a trabajar en el pseudocódigo del avance previo y porque se presta muy bien para mostrar el paradigma funcional.

El problema que resuelve es bastante concreto: evita que recepción o el paciente tengan que ponerse a revisar cita por cita para encontrar un hueco libre. En lugar de eso, la función toma la lista de citas y devuelve directamente una lista de intervalos disponibles.

Los usuarios que la consultarían son Recepción y Paciente, dentro del caso de uso CU03. Para esta etapa usamos una agenda ficticia como entrada; todavía no hace falta base de datos.

2.2 Relación con requerimientos y casos de uso

Elemento	Relación
R04	El resultado principal de la funcionalidad son los horarios disponibles que el sistema muestra al usuario.
R06	La disponibilidad calculada evita seleccionar un horario que ya esté ocupado por otra cita.
CU03	Es el caso de uso principal: consultar horarios disponibles.
CU04	La información que produce esta funcionalidad se puede usar después para agendar una cita.

2.3 Entradas, transformaciones y salidas

Etapa	Contenido
Entradas	Lista de citas con inicio y duración; horario laboral; duración solicitada para una nueva cita.
Validación	Se comprueba que la duración sea positiva y que las citas existentes permanezcan dentro del horario laboral.

Etapa	Contenido
Transformación 1	Cada cita se convierte en un intervalo [inicio, fin].
Transformación 2	Los intervalos se ordenan por inicio y los que se traslapan se fusionan.
Transformación 3	Se calculan los huecos entre los intervalos ocupados.
Transformación 4	Cada hueco se divide en bloques consecutivos de la duración solicitada.
Salida	Lista de intervalos disponibles o un resultado de error.

2.4 Adecuación al paradigma funcional

Esta funcionalidad se presta bastante al paradigma funcional porque se puede ver como una cadena de transformaciones de datos. Cada función recibe lo que necesita de manera explícita y devuelve estructuras nuevas, sin depender de nada externo ni modificar lo que le llegó. Eso hace más fácil razonar sobre cada paso por separado y reutilizar funciones pequeñas en otros lados.

Comparado con una solución imperativa, nos parece que aquí ganamos en varias cosas: casi no usamos variables mutables, se ve claro dónde termina una transformación y empieza otra, podemos usar composición y funciones de orden superior, y cada función se puede probar sola sin montar todo el flujo. Como desventaja, crear colecciones nuevas en cada paso puede pesar más en memoria si los datos fueran enormes, y el estilo puede resultar raro para quien siempre ha programado con ciclos y variables que se van modificando. Para esta etapa, con datos ficticios y pequeños, eso no representa un problema real.

3. Diseño de la solución

3.1 Estructuras inmutables

Se utilizan tres estructuras principales. Cita representa una cita existente; Intervalo representa un periodo de tiempo; Resultado permite devolver Éxito o Error. En Python se implementan con dataclasses congeladas mediante @dataclass(frozen=True), evitando la modificación posterior de sus atributos.

Tipo	Campos	Propósito
Cita	inicio, duracion	Representar una cita de la agenda.
Intervalo	inicio, fin	Representar tiempo ocupado o disponible.
Resultado	Exito(valor) / Error(mensaje)	Comunicar una salida válida o un error sin excepciones para reglas esperadas.

3.2 Funciones, entradas y salidas

Función	Entrada	Salida
calcular_fin	Cita	Entero
validar_cita	Cita, Intervalo	Resultado<cita>
cita_a_intervalo	Cita	Intervalo

Función	Entrada	Salida
ordenar_por_inicio	Lista de intervalos	Nueva lista ordenada
fusionar	Lista de intervalos	Nueva lista fusionada
calcular_huecos	Lista de intervalos, horario	Lista de huecos
dividir_hueco	Intervalo, duración	Lista de intervalos
formatear_hora	Minutos	Texto HH:MM
buscar_disponibilidad	Citas, horario, duración	Resultado<lista<intervalo>>

3.3 Pseudocódigo

```
TIPO Cita = (inicio: Entero, duracion: Entero)

TIPO Intervalo = (inicio: Entero, fin: Entero)

TIPO Resultado = Exito(valor) | Error(mensaje)

CONSTANTE HORARIO_LABORAL = Intervalo(480, 840)


FUNCION calcularFin(cita) -> Entero

    RETORNAR cita.inicio + cita.duracion

FIN FUNCION


FUNCION validarCita(cita, horario) -> Resultado

    SI cita.duracion <= 0 ENTONCES

        RETORNAR Error("Duracion invalida")

    SI cita.inicio < horario.inicio O calcularFin(cita) > horario.fin ENTONCES

        RETORNAR Error("Cita fuera del horario laboral")

    RETORNAR Exito(cita)

FIN FUNCION


FUNCION citaAIntervalo(cita) -> Intervalo

    RETORNAR Intervalo(cita.inicio, calcularFin(cita))

FIN FUNCION


FUNCION ordenarPorInicio(intervalos) -> Lista

    RETORNAR copiaOrdenada(intervalos, clave = lambda i -> i.inicio)

FIN FUNCION


FUNCION fusionar(intervalos) -> Lista

    FUNCION acumular(acc, actual) -> Lista

        SI acc esta vacia ENTONCES

            RETORNAR [actual]
```

```
        ultimo <- ultimo elemento de acc
        SI actual.inicio <= ultimo.fin ENTONCES
            combinado <- Intervalo(ultimo.inicio, max(ultimo.fin, actual.fin))
            RETORNAR (acc sin ultimo) + [combinado]
        RETORNAR acc + [actual]
    FIN FUNCION
    RETORNAR reducir(intervalos, acumular, [])
FIN FUNCION

FUNCION calcularHuecos(ocupados, horario) -> Lista
    puntos <- [Intervalo(horario.inicio, horario.inicio)] + ocupados
              + [Intervalo(horario.fin, horario.fin)]
    pares <- zip(puntos sin ultimo, puntos sin primero)
    candidatos <- mapear(pares, lambda (a,b) -> Intervalo(a.fin, b.inicio))
    RETORNAR filtrar(candidatos, lambda h -> h.fin > h.inicio)
FIN FUNCION

FUNCION dividirHueco(hueco, duracion) -> Lista
    cantidad <- division_entera(hueco.fin - hueco.inicio, duracion)
    RETORNAR mapear(rango(0, cantidad),
                    lambda k -> Intervalo(hueco.inicio + k*duracion,
                                          hueco.inicio + (k+1)*duracion))
FIN FUNCION

FUNCION buscarDisponibilidad(citas, horario, duracion) -> Resultado
    validar duracion
    validaciones <- mapear(citas, lambda c -> validarCita(c, horario))
    SI existe error EN validaciones -> RETORNAR primer error
    espacios <- componer(
        lambda cs -> mapear(cs, citaIntervalo),
```

```
        ordenarPorInicio,
        fusionar,
        lambda ocs -> calcularHuecos(ocs, horario),
        lambda hs -> aplanar(mapear(hs, lambda h -> dividirHueco(h, duracion)))
    )(citas)
    RETORNAR Exito(espacios)
FIN FUNCION
```

3.4 Conceptos funcionales aplicados

- Funciones puras: el resultado depende solo de las entradas, no de estado externo.
- Inmutabilidad: las citas y los intervalos originales no se modifican; siempre se producen colecciones nuevas.
- Funciones de orden superior: map, filter, reduce, existe y componer reciben funciones como parámetro.
- Lambdas: las usamos como claves, filtros y transformaciones pequeñas.
- Composición: buscar_disponibilidad encadena varias etapas de transformación.
- Separación cálculo / E-S: la lógica de negocio quedó aparte de la impresión y de la interfaz web.

4. Implementación

4.1 Organización del proyecto

```
gestor_citas_project/
├── app.py
├── INICIAR.bat
├── abrir_frontend.bat
├── ejecutar_pruebas.bat
├── EJECUTAR.txt
├── README.md
├── data/
│   └── citas_ficticias.json
├── frontend/
│   ├── index.html
│   ├── styles.css
│   └── app.js
├── src/
│   ├── __init__.py
│   ├── gestor_citas.py
│   └── servidor.py
├── tests/
```

```
│   ├── __init__.py
│   └── test_gestor_citas.py
└── docs/
```

4.2 Ejecución en Python y Visual Studio Code

1. Abrir la carpeta completa del proyecto en Visual Studio Code.
2. Abrir Terminal > Nueva terminal.
3. Ejecutar python app.py. En Windows también se puede ejecutar py app.py.
4. El programa inicia un servidor local y abre el navegador en una dirección 127.0.0.1, buscando automáticamente otro puerto si el 8000 ya está ocupado.
5. En VS Code también existe una configuración de ejecución llamada “Gestor de Citas - Web”.
6. Las pruebas se ejecutan con python -m unittest discover -s tests -v o mediante la configuración “Pruebas - unittest”.

4.3 Frontend desarrollado

Armamos una interfaz web sin dependencias externas. El frontend usa HTML, CSS y JavaScript puro, y presenta un flujo sencillo para consultar disponibilidad: se elige médico, fecha y duración, se ejecuta la búsqueda y se muestran las citas ocupadas junto con los horarios disponibles para seleccionar uno.

La interfaz consume dos rutas del servidor: GET /api/doctores para traer el catálogo ficticio y GET /api/disponibilidad?doctor=...&duracion=... para pedir el cálculo que hace Python.

Como respaldo para pruebas de interfaz, index.html también tiene un modo de demostración local. Si se abre directamente o con Live Server y no hay conexión con Python, se cargan los mismos datos ficticios y el frontend puede mostrar resultados sin depender del servidor.

4.4 Datos ficticios

ID	Médico	Especialidad	Citas iniciales
MED001	Dra. Ana Lopez	Medicina general	09:00-09:30; 10:00-11:00; 10:30-11:00
MED002	Dr. Carlos Martinez	Cardiología	08:00-08:30; 09:30-10:00; 12:00-13:00
MED003	Dra. Sofia Hernandez	Pediatría	Sin citas

5. Pruebas

Este apartado documenta las pruebas realizadas al frontend del Gestor de Citas Médicas, abierto con el archivo abrir_frontend.bat. Ese archivo abre frontend/index.html directamente en el navegador, sin servidor de Python, por lo que la aplicación trabaja en modo demostración con la lógica de app.js (función fallbackAvailability).

Metodología

- **Cantidad:** 20 casos de prueba.

- **Tipos de caso:** Normales (P01–P03, P13), límites (P04, P07, P09), datos vacíos (P03, P15, P16, P19) e inválidos (P05, P06, P08, P18), además de casos de interfaz (P12–P19).
- **Resultado esperado antes de ejecutar:** Los valores esperados se calcularon a mano (horario laboral 08:00–14:00, es decir 480–840 minutos) y se registraron en la Fase 1 antes de correr cualquier prueba (Figura 1).
- **Ejecución:** La Fase 2 ejecuta cada caso contra el código real, registra el resultado obtenido y lo compara con el esperado para marcarlo como Aprobada o Fallida (Figura 2). Los casos de interfaz se verificaron además en un navegador real (Chromium) abriendo index.html como archivo local, con una captura por caso.

Los resultados completos se presentan en la tabla de la siguiente página.

Casos de prueba y resultados

ID	Escenario	Entrada	Resultado esperado	Resultado obtenido	Estado
P01	Disponibilidad MED001, bloques de 30 min (normal)	citas 540/30, 600/60, 630/30; duración 30	n=9; primero=08:00-08:30; ultimo=13:30-14:00	n=9; primero=08:00-08:30; ultimo=13:30-14:00	**Aprobada**
P02	Disponibilidad MED002, bloques de 30 min (normal)	citas 480/30, 570/30, 720/60; duración 30	n=8; primero=08:30-09:00; ultimo=13:30-14:00	n=8; primero=08:30-09:00; ultimo=13:30-14:00	**Aprobada**
P03	Agenda vacía (dato vacío), bloques de 60 min	citas = []; duración 60	n=6; primero=08:00-09:00; ultimo=13:00-14:00	n=6; primero=08:00-09:00; ultimo=13:00-14:00	**Aprobada**
P04	Duración cero (límite inválido)	citas = []; duración 0	error=Duracion invalida	error=Duracion invalida	**Aprobada**
P05	Duración negativa (inválida)	citas = []; duración -15	error=Duracion invalida	error=Duracion invalida	**Aprobada**
P06	Duración mayor que la jornada (361 min)	citas = []; duración 361	error=La duracion excede el horario laboral	error=La duracion excede el horario laboral	**Aprobada**
P07	Duración igual a la jornada completa (límite válido)	citas = []; duración 360	n=1; primero=08:00-14:00; ultimo=08:00-14:00	n=1; primero=08:00-14:00; ultimo=08:00-14:00	**Aprobada**
P08	Cita que se pasa del cierre (inicio 830, dur 20 → fin 850)	citas 830/20; duración 30	error=Cita fuera del horario laboral	error=Cita fuera del horario laboral	**Aprobada**
P09	Cita que termina justo al cierre (límite válido, fin = 840)	citas 810/30; duración 30	n=11; primero=08:00-08:30; ultimo=13:00-13:30	n=11; primero=08:00-08:30; ultimo=13:00-13:30	**Aprobada**
P10	Citas traslapadas (540-600 y 570-600)	citas 540/60, 570/30; duración 30	n=10; primero=08:00-08:30; ultimo=13:30-14:00	n=10; primero=08:00-08:30; ultimo=13:30-14:00	**Aprobada**
P11	Citas desordenadas: resultado correcto y lista de entrada sin cambios	citas [630/30, 540/30]; duración 30	n=10; primero=08:00-08:30; ultimo=13:30-14:00; entradaIntacta=true	n=10; primero=08:00-08:30; ultimo=13:30-14:00; entradaIntacta=true	**Aprobada**
P12	Abrir el frontend sin servidor: carga de médicos en modo demostración	abrir index.html; fetch("/api/doctores") falla	medicos=3; primero=MED001; conexion=Modo demostración: frontend sin servidor.	medicos=3; primero=MED001; conexion=Modo demostración: frontend sin servidor.	**Aprobada**
P13	UI: buscar MED001, 30 min, fecha 2026-10-05	médico MED001; duración 30; fecha 2026-10-05; clic en "Buscar horarios"	horarios=9; contador=9 disponibles; ocupadas=3; mensaje=Fecha: 2026-10-05 · Duración: 30 minutos · Modo demostración	horarios=9; contador=9 disponibles; ocupadas=3; mensaje=Fecha: 2026-10-05 · Duración: 30 minutos · Modo demostración	**Aprobada**
P14	UI: MED001 con 120 min (solo cabe un hueco)	médico MED001; duración 120	horarios=1; contador=1 disponibles; inicioPrimero=11:00	horarios=1; contador=1 disponibles; inicioPrimero=11:00	**Aprobada**
P15	UI: médico sin citas (MED003, agenda vacía), 30 min	médico MED003; duración 30	horarios=12; ocupadas=0; textoOcupadas=No hay citas ocupadas. Todo el horario laboral está libre.	horarios=12; ocupadas=0; textoOcupadas=No hay citas ocupadas. Todo el horario laboral está libre.	**Aprobada**
P16	UI: fecha vacía (dato vacío)	médico MED001; duración 30; fecha = ""	mensaje=Fecha: sin fecha · Duración: 30 minutos · Modo demostración; horarios=9	mensaje=Fecha: sin fecha · Duración: 30 minutos · Modo demostración; horarios=9	**Aprobada**

ID	Escenario	Entrada	Resultado esperado	Resultado obtenido	Estado
P17	UI: seleccionar un horario con clic	MED001, 30 min; clic en el primer horario (08:00)	mensaje=Horario seleccionado: 08:00.; marcado=true	mensaje=Horario seleccionado: 08:00.; marcado=true	**Aprobada**
P18	UI: médico inexistente (dato inválido)	médico = "MED999"; duración 30	mensajeDeError=true; horarios=0; botonHabilitado=true	mensajeDeError=true; horarios=0; botonHabilitado=true	**Aprobada**
P19	UI: sin médico seleccionado (valor vacío)	médico = ""; clic en "Buscar horarios"	mensaje=Los horarios disponibles aparecerán aquí.; horarios=0; botonHabilitado=true	mensaje=Los horarios disponibles aparecerán aquí.; horarios=0; botonHabilitado=true	**Aprobada**
P20	Los datos originales no se modifican tras todas las pruebas	SHA-256 de app.js, index.html y data/citas_ficticias.json antes/después; FALLBACK_DOCTORS	archivosIguales=true; fallbackDoctorsIguales=true	archivosIguales=true; fallbackDoctorsIguales=true	**Aprobada**

Evidencia de ejecución

Primero se muestran las capturas generales (definición de esperados y ejecución). Después hay una captura por cada caso, de P01 a P20, con el ID del caso visible en la parte superior de la imagen. Los casos de interfaz son capturas reales de index.html en Chromium; los casos de lógica pura (P04 a P11) muestran la llamada a fallbackAvailability ejecutada en la consola del navegador, y P20 los hashes antes y después.

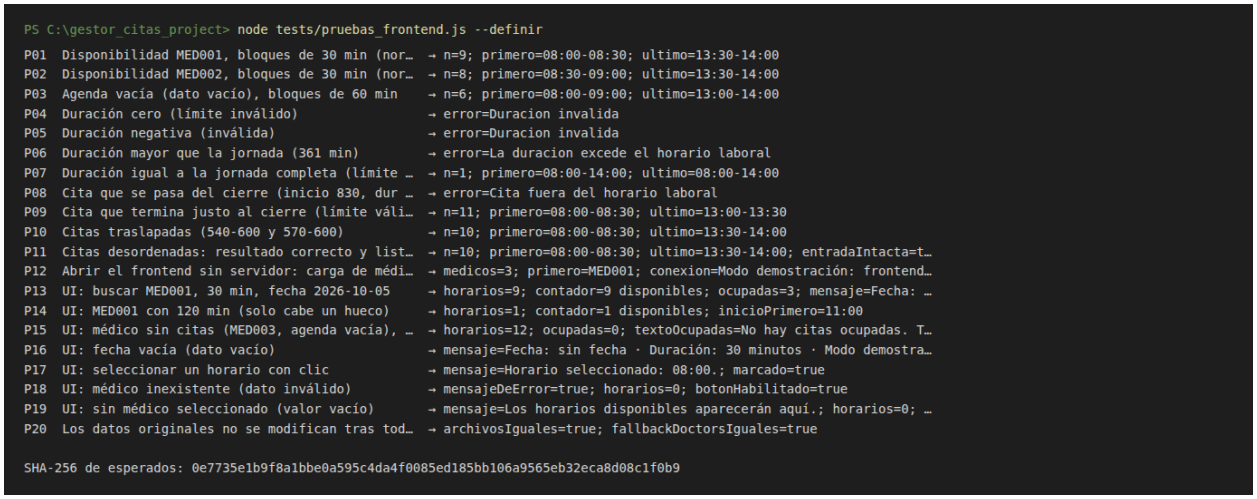


Figura 1. *Fase 1: resultados esperados definidos antes de ejecutar (comando --definir).*

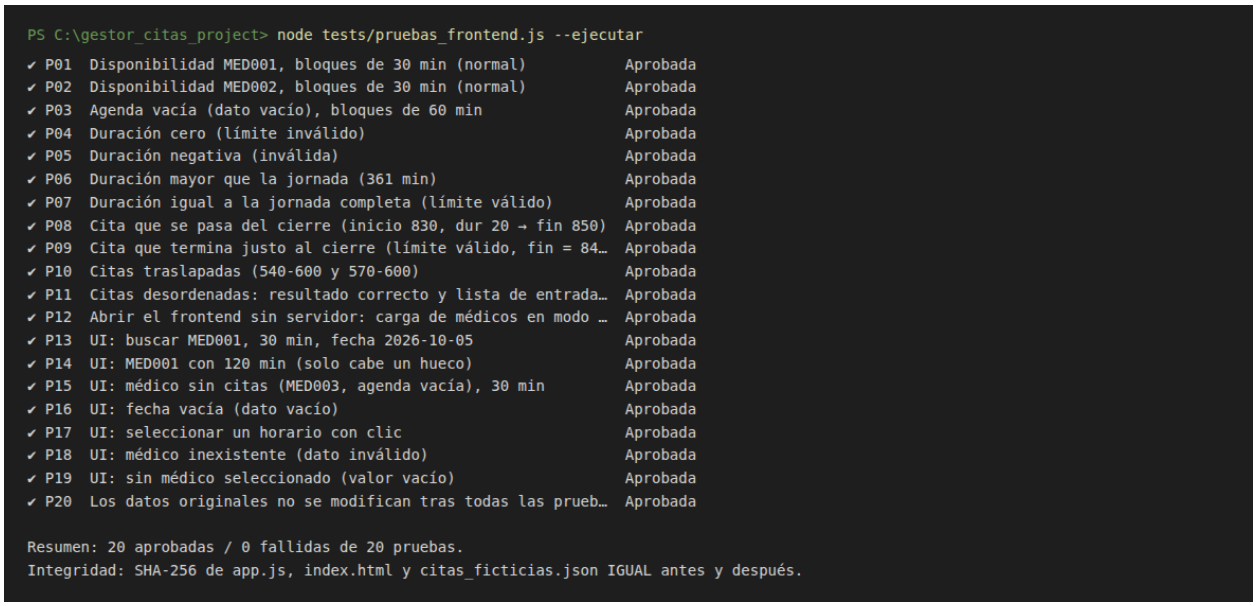


Figura 2. *Fase 2: ejecución de las 20 pruebas y comparación contra lo esperado (comando --ejecutar).*

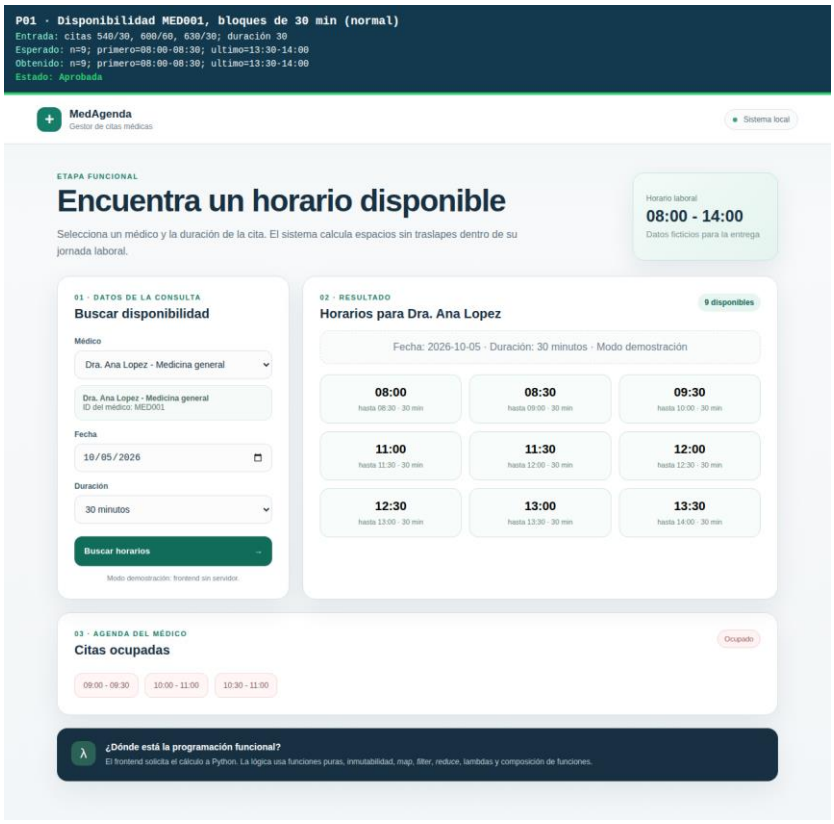


Figura 3. *Caso P01 – Disponibilidad MED001, bloques de 30 min (normal)*
Estado: Aprobada

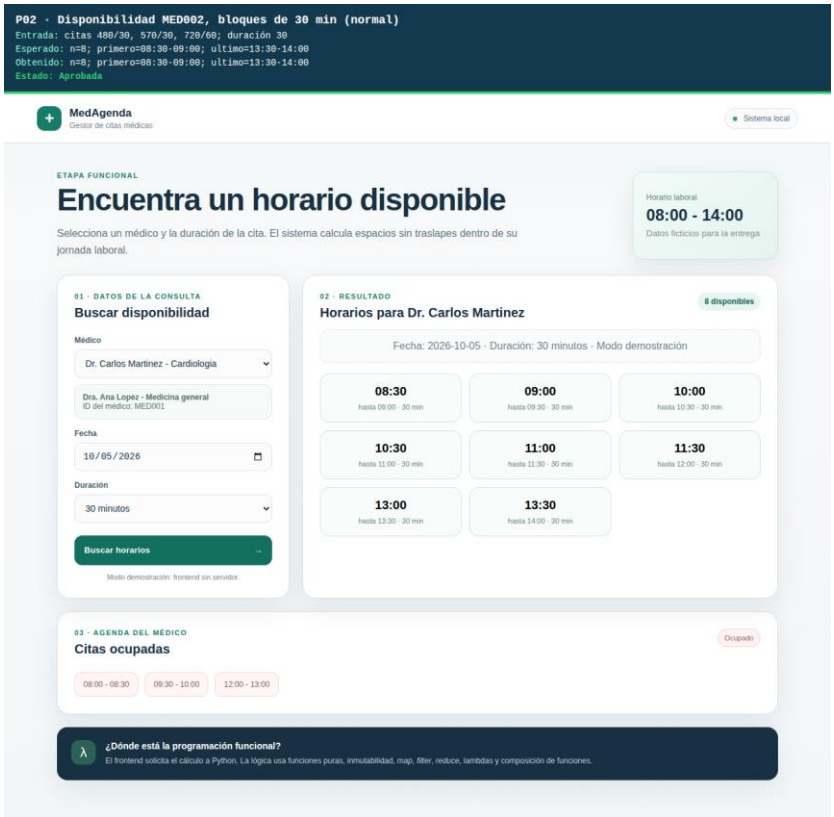


Figura 4. *Caso P02 – Disponibilidad MED002, bloques de 30 min (normal)*
Estado: Aprobada

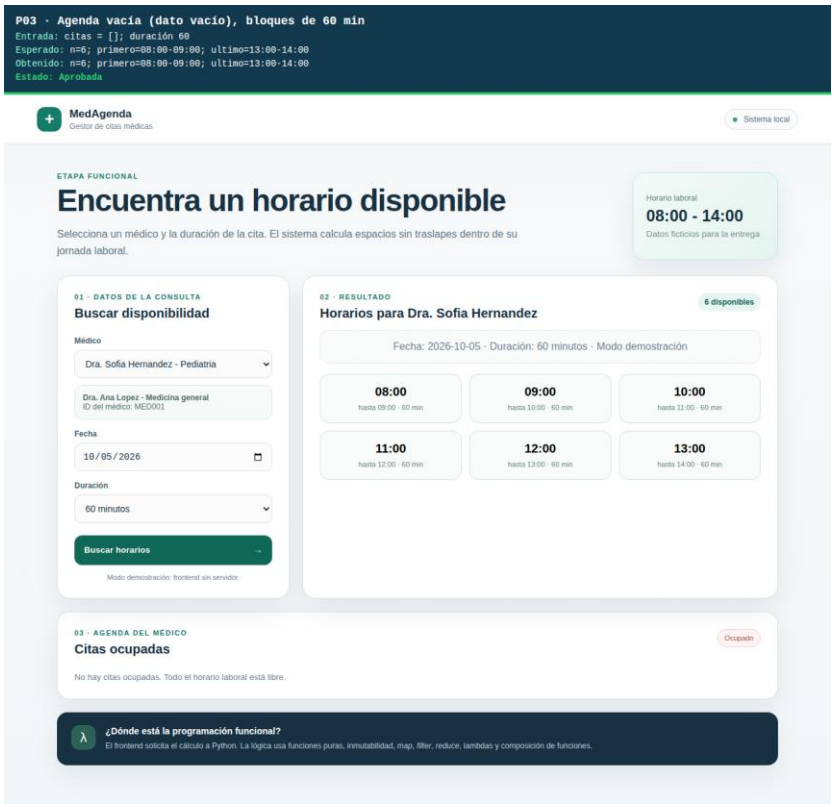


Figura 5. *Caso P03 – Agenda vacía (dato vacío), bloques de 60 min*
Estado: Aprobada

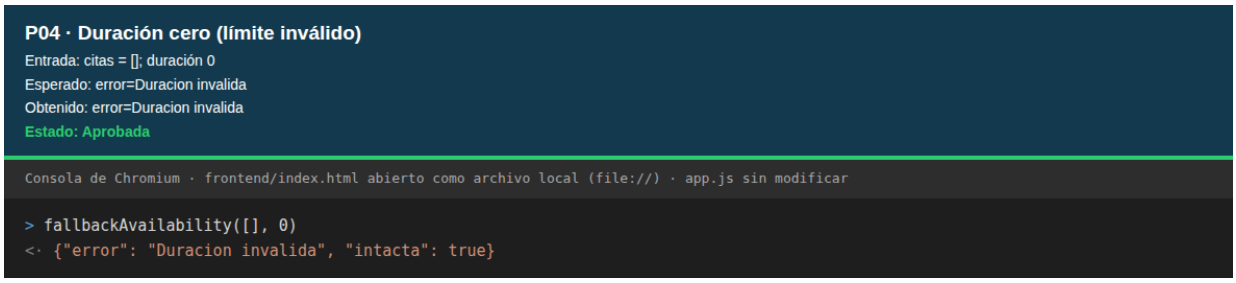


Figura 6. *Caso P04 – Duración cero (límite inválido)*
Estado: Aprobada

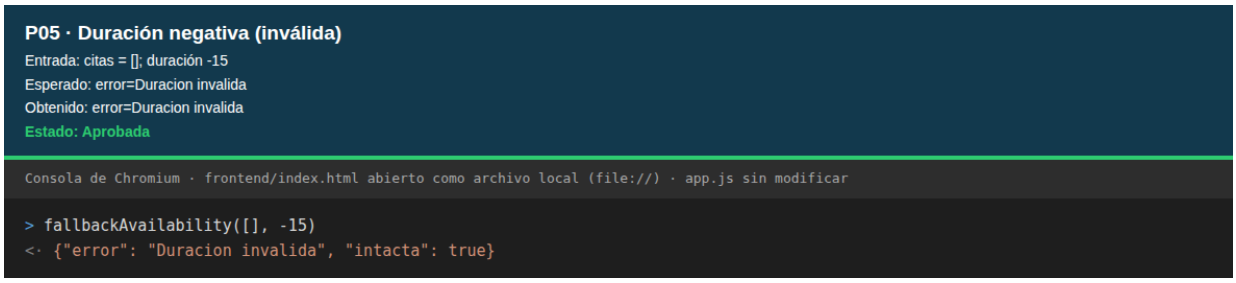


Figura 7. *Caso P05 – Duración negativa (inválida)*
Estado: Aprobada



Figura 8. *Caso P06 – Duración mayor que la jornada (361 min)*
Estado: Aprobada



Figura 9. *Caso P07 – Duración igual a la jornada completa (límite válido)*
Estado: Aprobada



Figura 10. *Caso P08 – Cita que se pasa del cierre (inicio 830, dur 20 → fin 850)*
Estado: Aprobada

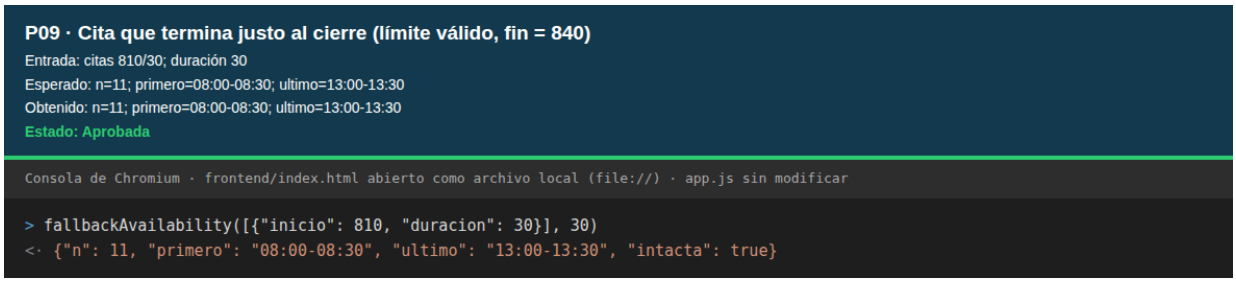


Figura 11. *Caso P09 – Cita que termina justo al cierre (límite válido, fin = 840)*
Estado: Aprobada

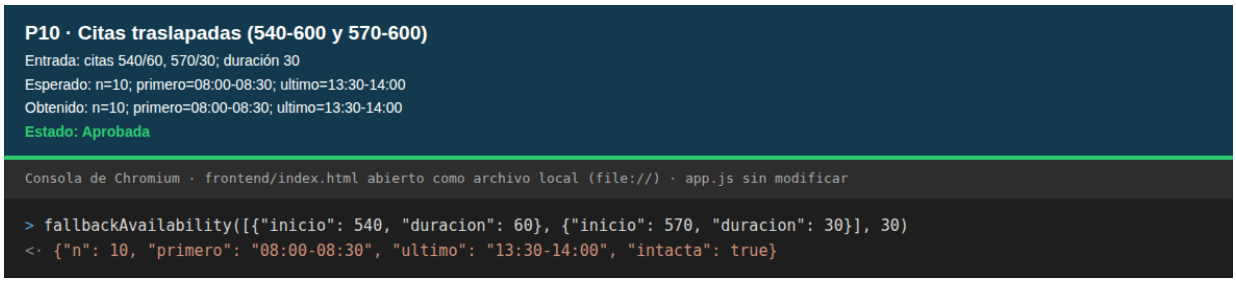


Figura 12. *Caso P10 – Citas traslapadas (540-600 y 570-600)*
Estado: Aprobada

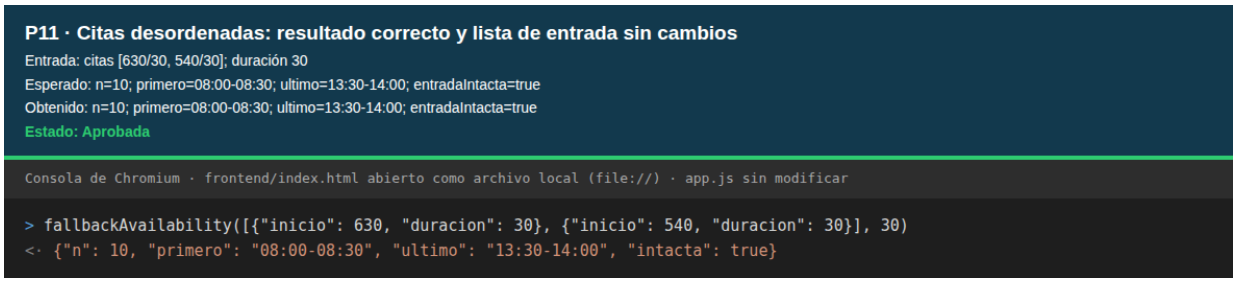


Figura 13. *Caso P11 – Citas desordenadas: resultado correcto y lista de entrada sin cambios*
Estado: Aprobada

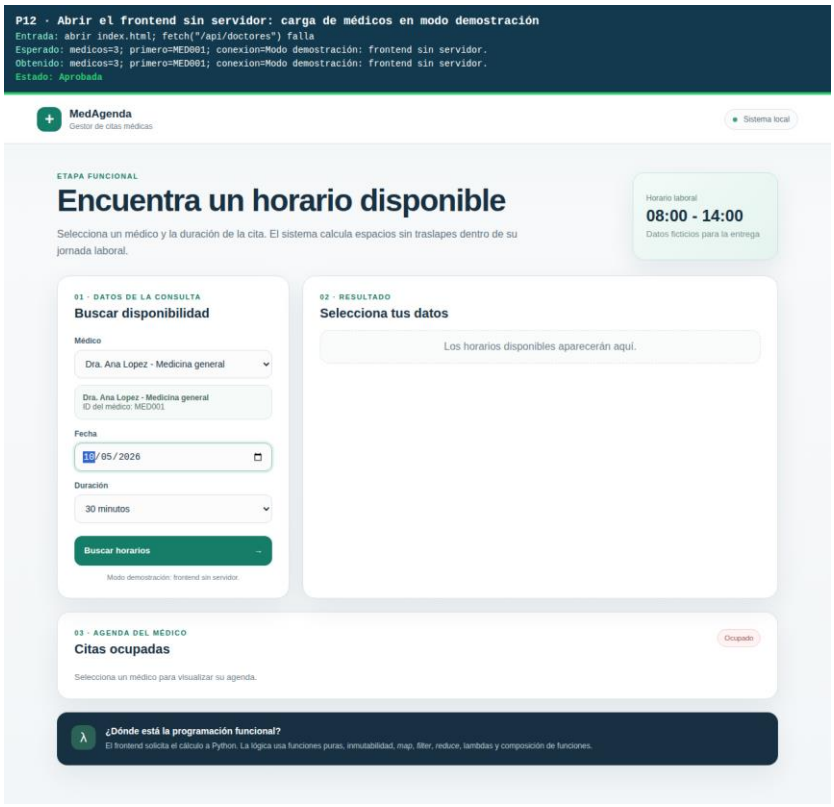


Figura 14. *Caso P12 – Abrir el frontend sin servidor: carga de médicos en modo demostración*
Estado: Aprobada

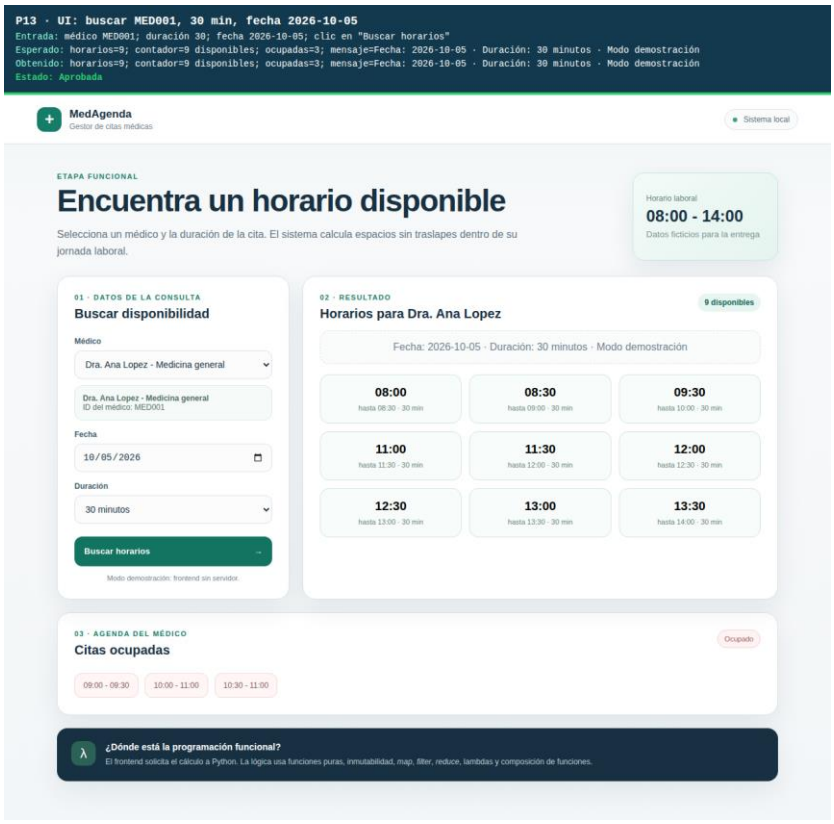


Figura 15. *Caso P13 – UI: buscar MED001, 30 min, fecha 2026-10-05*
Estado: Aprobada

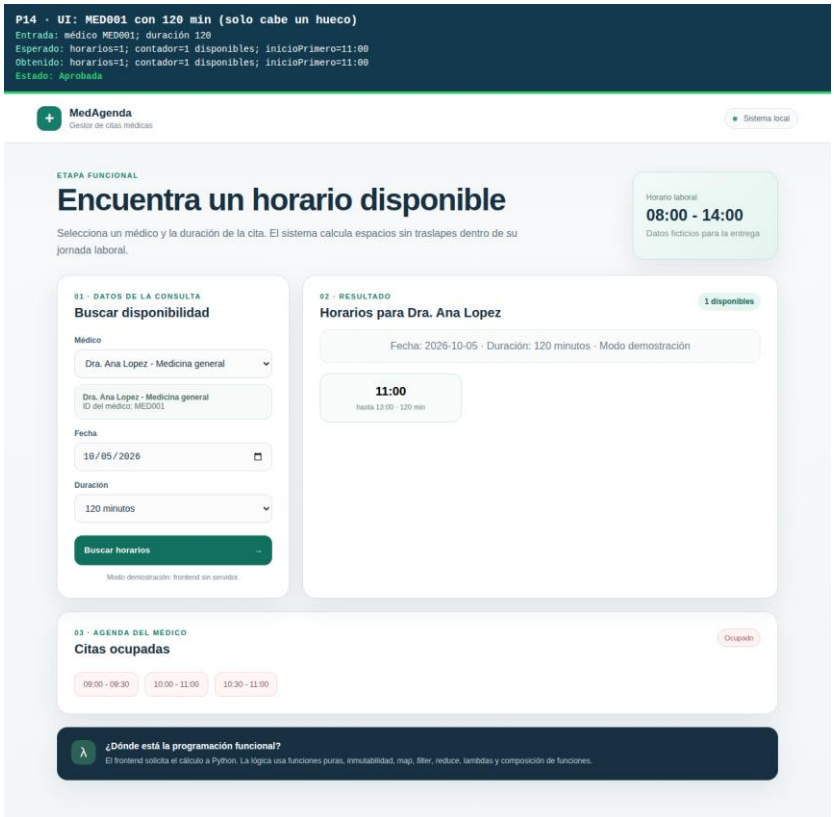


Figura 16. *Caso P14 – UI: MED001 con 120 min (solo cabe un hueco)*
Estado: Aprobada

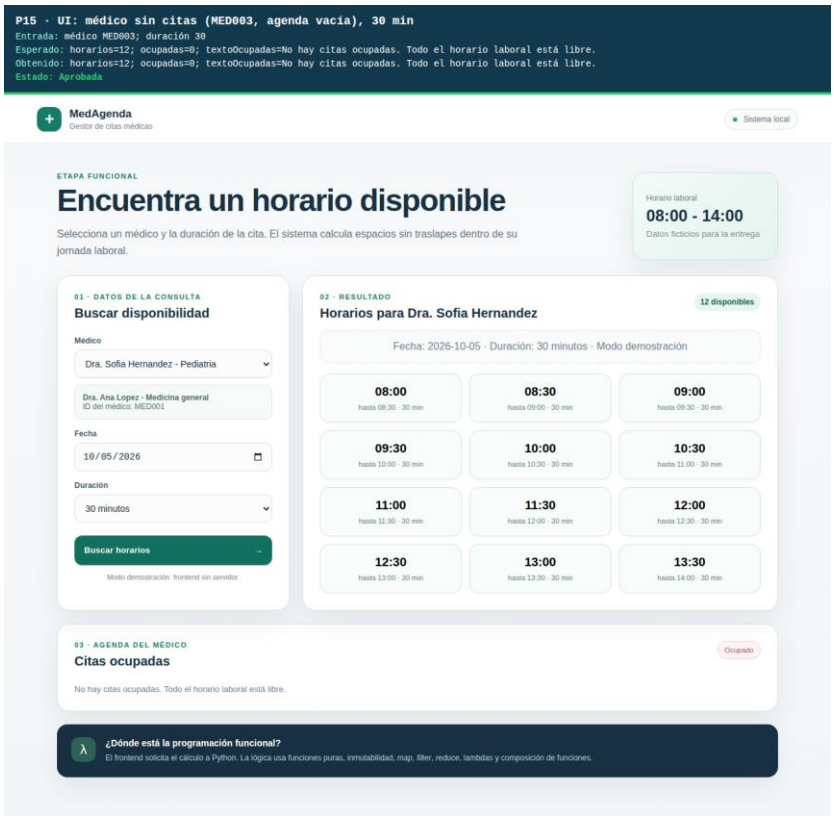


Figura 17. *Caso P15 – UI: médico sin citas (MED003, agenda vacía), 30 min*
Estado: Aprobada

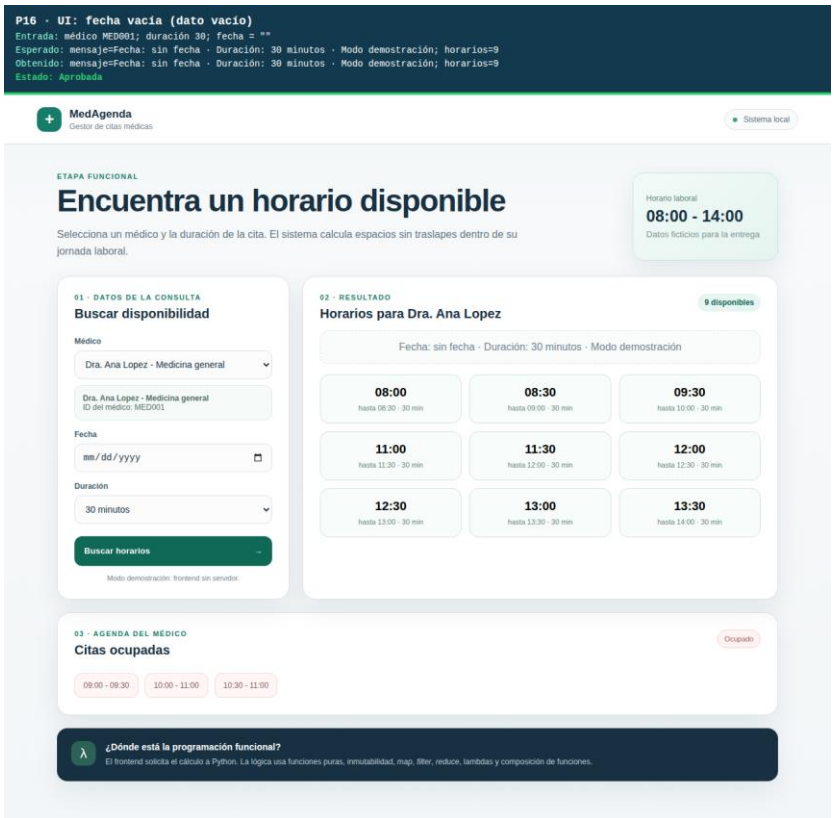


Figura 18. *Caso P16 – UI: fecha vacía (dato vacío)*
Estado: Aprobada

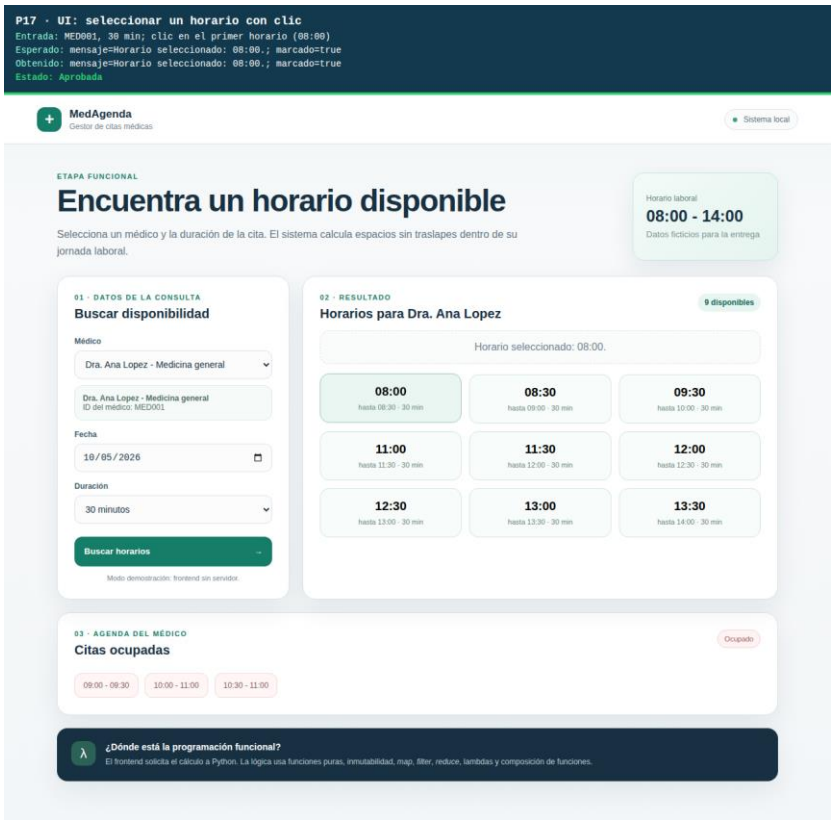


Figura 19. *Caso P17 – UI: seleccionar un horario con clic*
Estado: Aprobada

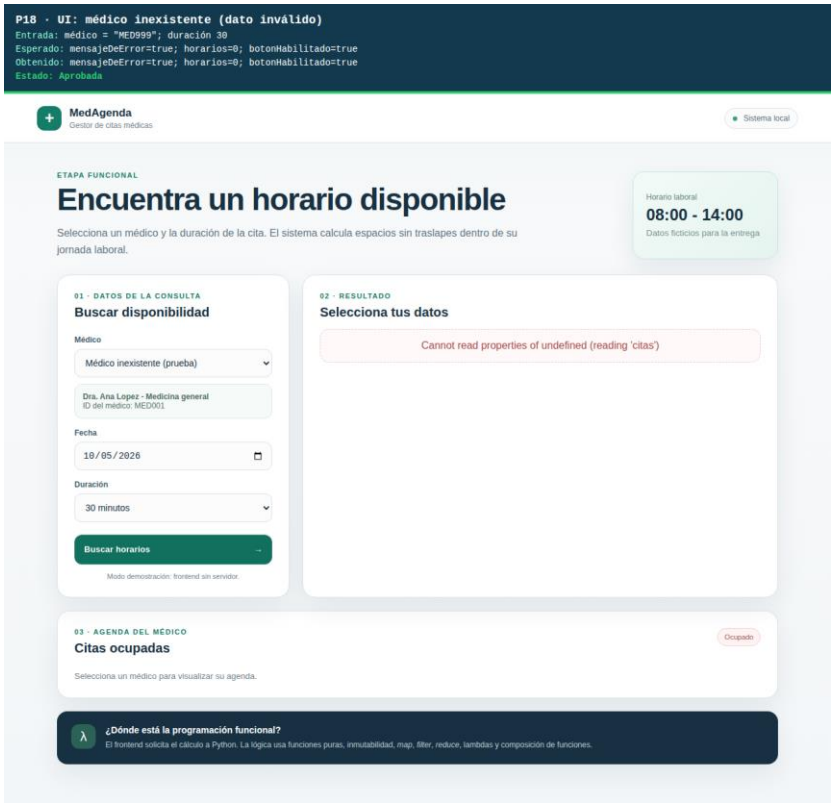


Figura 20. *Caso P18 – UI: médico inexistente (dato inválido)*
Estado: Aprobada

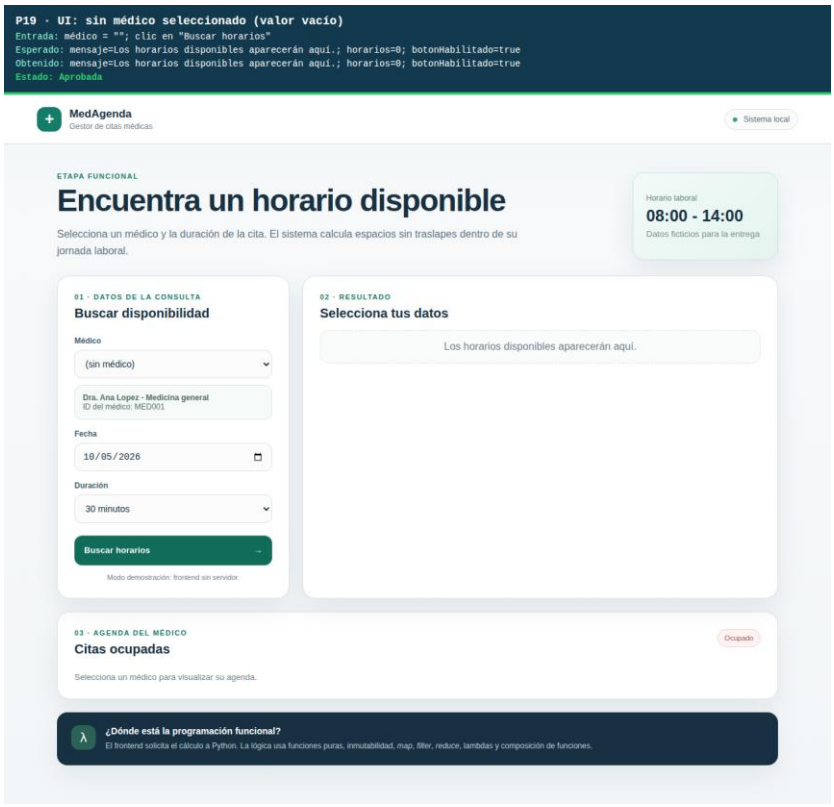


Figura 21. *Caso P19 – UI: sin médico seleccionado (valor vacío)*
Estado: Aprobada

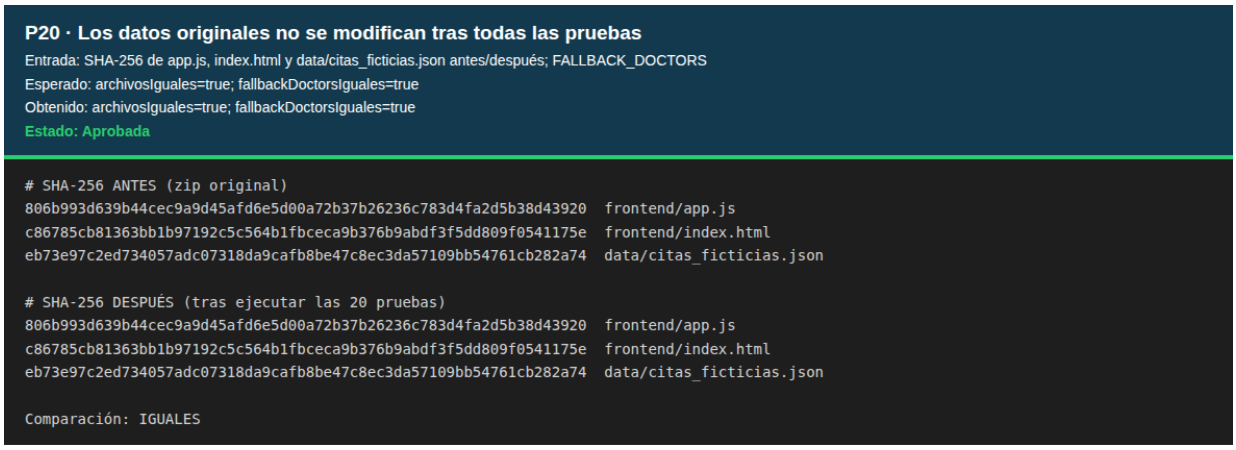


Figura 22. *Caso P20 – Los datos originales no se modifican tras todas las pruebas*
Estado: Aprobada

5.1 Resultado global

Las 20 pruebas resultaron aprobadas: El frontend calcula correctamente los horarios disponibles, rechaza duraciones y citas inválidas, maneja agendas vacías y datos incompletos, y no altera los datos originales.

6. Repositorio: https://github.com/mgKrisz10/Gestor_Hospital



7. Conclusiones

La funcionalidad de búsqueda de horarios disponibles quedó diseñada e implementada con un flujo que va desde los requerimientos y casos de uso hasta el pseudocódigo y el código en Python. La solución se apoya en inmutabilidad y composición funcional para transformar la agenda del médico en intervalos disponibles.

Agregar el frontend nos ayudó a ver el resultado de forma clara sin romper la separación del cálculo. La interfaz puede consumir el backend en Python o usar el modo de demostración con datos ficticios, mientras que la lógica principal sigue concentrada en el módulo funcional y se puede verificar con las 15 pruebas automatizadas.

Anexo A. Fragmento principal de implementación

```
def calcular_fin(cita: Cita) -> int:
    return cita.inicio + cita.duracion


def validar_cita(cita: Cita, horario: Intervalo) -> Resultado:
    if cita.duracion <= 0:
        return Error("Duracion invalida")
    if cita.inicio < horario.inicio or calcular_fin(cita) > horario.fin:
        return Error("Cita fuera del horario laboral")
    return Exito(cita)


def cita_a_intervalo(cita: Cita) -> Intervalo:
    return Intervalo(cita.inicio, calcular_fin(cita))


def ordenar_por_inicio(intervalos: Sequence[Intervalo]) -> Tuple[Intervalo, ...]:
    return tuple(sorted(intervalos, key=lambda intervalo: intervalo.inicio))


def fusionar(intervalos: Sequence[Intervalo]) -> Tuple[Intervalo, ...]:
    def acumular(
        acumulados: Tuple[Intervalo, ...], actual: Intervalo
    ) -> Tuple[Intervalo, ...]:
        if not acumulados:
```

```
            return (actual,)
        ultimo = acumulados[-1]
        if actual.inicio <= ultimo.fin:
            combinado = Intervalo(ultimo.inicio, max(ultimo.fin, actual.fin))
            return acumulados[:-1] + (combinado,)
        return acumulados + (actual,)
    return reduce(acumular, intervalos, ())


def calcular_huecos(
    ocupados: Sequence[Intervalo], horario: Intervalo
) -> Tuple[Intervalo, ...]:
    puntos = (
        Intervalo(horario.inicio, horario.inicio),
        *ocupados,
        Intervalo(horario.fin, horario.fin),
    )
    pares = zip(puntos[:-1], puntos[1:])
    candidatos = tuple(
        map(lambda par: Intervalo(par[0].fin, par[1].inicio), pares)
    )
    return tuple(filter(lambda hueco: hueco.fin > hueco.inicio, candidatos))


def dividir_hueco(hueco: Intervalo, duracion: int) -> Tuple[Intervalo, ...]:
    cantidad = (hueco.fin - hueco.inicio) // duracion
    return tuple(
        map(
            lambda k: Intervalo(
                hueco.inicio + k * duracion,
                hueco.inicio + (k + 1) * duracion,
            ),
            range(cantidad),
        )
    )


def formatear_hora(minutos: int) -> str:
    horas, mins = divmod(minutos, 60)
    return f"{horas:02d}:{mins:02d}"


def componer(*funciones: Callable) -> Callable:
    def pipeline(valor):
        return reduce(lambda actual, funcion: funcion(actual), funciones, valor)
    return pipeline


def buscar_disponibilidad(
    citas: Sequence[Cita], horario: Intervalo, duracion: int
```

```
) -> Resultado:
    """Calcula bloques disponibles sin modificar las citas de entrada."""
    if duracion <= 0:
        return Error("Duracion invalida")
    if duracion > horario.fin - horario.inicio:
        return Error("La duracion excede el horario laboral")

    validaciones = tuple(map(lambda cita: validar_cita(cita, horario), citas))
    errores = tuple(filter(lambda r: isinstance(r, Error), validaciones))
    if errores:
        return errores[0]

    pipeline = componer(
        lambda cs: tuple(map(cita_a_intervalo, cs)),
        ordenar_por_inicio,
        fusionar,
        lambda ocupadas: calcular_huecos(ocupadas, horario),
        lambda huecos: tuple(
            chain.from_iterable(
                map(lambda hueco: dividir_hueco(hueco, duracion), huecos)
            )
        ),
    )
    return Exito(pipeline(tuple(citas)))


def intervalo_a_dict(intervalo: Intervalo) -> dict:
    return asdict(intervalo)


def resultado_a_dict(resultado: Resultado) -> dict:
    if isinstance(resultado, Error):
        return {"ok": False, "error": resultado.mensaje}
    return {
        "ok": True,
        "intervalos": [intervalo_a_dict(i) for i in resultado.valor],
        "horario": intervalo_a_dict(HORARIO_LABORAL),
    }
```

8. Segundo parcial: alcance del MVP

El segundo parcial continúa la entrega funcional del Gestor de Citas Médicas. El cálculo de horarios libres basado en funciones puras se conserva. En esta etapa se integra una interfaz con inicio de sesión, almacenamiento SQLite y un conjunto de permisos por tipo de usuario.

La tabla de requerimientos R01–R14 de la primera entrega presenta objetivos del sistema completo; no implica que todos estén implementados. La siguiente lista define las acciones disponibles en el demo de este segundo parcial.

8.1 Funcionalidades demostrables

ID	Función implementada	Roles que pueden utilizarla
M01	Iniciar y cerrar sesión mediante usuario y contraseña	Los cinco perfiles de prueba
M02	Consultar catálogo de médicos y horarios por fecha	Todos (el médico solo consulta su propio médico)
M03	Agendar citas y registrar datos del paciente, sin empalmes	Recepción y administrador
M04	Cancelar una cita programada y liberar el horario	Recepción y administrador
M05	Consultar agenda de citas según los permisos	Recepción, administrador, doctor, paciente
M06	Registrar un médico nuevo y ver resumen/lista de cuentas	Solo administrador
M07	Consultar sin modificar citas ni ver datos personales	Personal autorizado

La cuenta de paciente de demostración muestra únicamente las citas que recepción vincule expresamente con ella. Los usuarios distintos de recepción y administrador no pueden registrar o cancelar una cita desde el servidor.

8.2 Relación con el primer parcial

R03 se demuestra mediante el inicio de sesión; R04, R05, R06 y R07 mediante la agenda; R09 y R10 mediante consultas filtradas; R02 de forma parcial mediante el alta de médicos. Las operaciones de administración completa de cuentas (R11), reprogramación (R08) y ausencia de doctor (R14) no son parte de la presente implementación.

9. Diseño del MVP: actores y permisos

El reporte original distingue cinco tipos de actores. Para mostrar su comportamiento, el demo incluye una cuenta por tipo; “personal autorizado” se utiliza como perfil de consulta restringida, aunque en un hospital real representa una categoría amplia de personal.

Actor del documento	Permisos dentro del MVP	Restricción comprobable
Administrador	Ver agenda y resumen, consultar usuarios, agendar/cancelar, registrar médicos	No se implementa editar/eliminar cuentas
Doctor	Consultar exclusivamente las citas del médico vinculado a su cuenta	No puede consultar agenda de otro médico ni modificar citas
Paciente	Consultar sus propias citas; consultar espacios de médicos	No ve citas de otros pacientes ni puede cancelarlas
Recepción	Consultar disponibilidad; registrar y cancelar citas	No administra usuarios ni agrega médicos
Personal autorizado	Consultar catálogo y disponibilidad sin datos personales	No puede consultar lista de citas ni registrar/cancelar

9.1 Flujo de uso

- El usuario selecciona un perfil de prueba o escribe sus credenciales y solicita el inicio de sesión.
- Python comprueba la contraseña y entrega un token temporal que identifica al usuario y su rol.
- El navegador muestra solo las secciones disponibles según el rol; cada solicitud al servidor también valida los permisos.
- Recepción elige médico, fecha y duración, selecciona un intervalo y registra una cita con un paciente ficticio.
- La cita queda persistida en SQLite. Si existe un empalme, el registro se rechaza; al cancelar se libera el intervalo.
- Doctor y paciente consultan agendas filtradas por sus vínculos. Administrador y personal autorizado ven los datos que su rol permite.

9.2 Credenciales solo para pruebas

Usuarios preconfigurados: admin, doctora, paciente, recepcion y consulta. Contraseña común de laboratorio: Demo2026!. No utilizar datos clínicos reales ni considerar estas cuentas seguras para producción.

10. Diagrama de clases del MVP

El diseño separa los modelos de negocio, la validación de la identidad, el cálculo de disponibilidad y el controlador HTTP. En el diagrama aparecen tanto las asociaciones de los registros como las dependencias entre servicios.

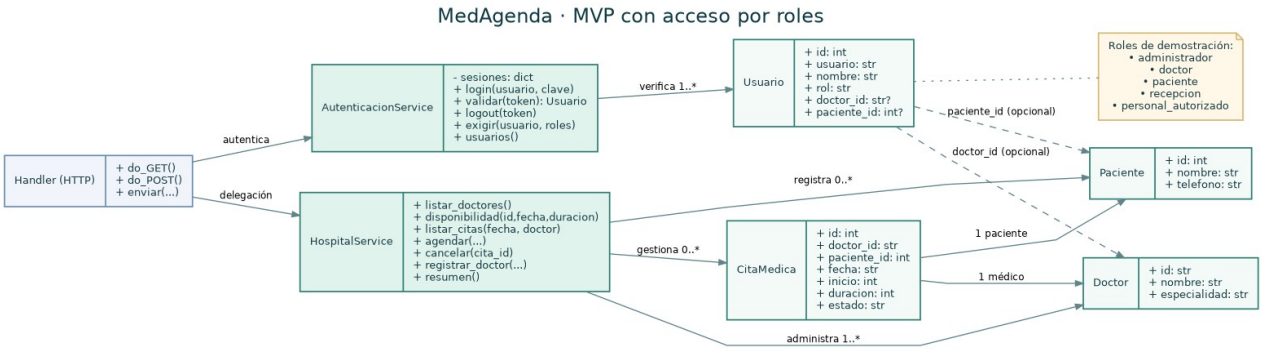


Figura 1. Diagrama actualizado: Usuario, AutenticacionService, HospitalService, Doctor, Paciente, CitaMedica y Handler.

10.1 Responsabilidad de cada componente

Clase o módulo	Responsabilidad
Usuario	Representa una cuenta, su rol y las referencias opcionales a un doctor o paciente.
AutenticacionService	Valida contraseñas, genera tokens, recupera la sesión y comprueba los roles.
HospitalService	Gestiona médicos, pacientes, citas, conflictos, cancelación y consultas en SQLite.
Cita / Intervalo	Estructuras inmutables utilizadas por la lógica funcional del primer parcial.
Handler	Recibe solicitudes HTTP, valida la sesión y delega las operaciones al servicio.

El servidor aplica el control de acceso en cada operación: ocultar un botón en HTML no constituye una medida suficiente.

11. Implementación y estructura

El programa se ejecuta localmente con Python 3.10+ y sin paquetes externos para el servidor. El frontend utiliza HTML, CSS y JavaScript; los datos de citas y usuarios se almacenan en SQLite.

Archivo	Contenido
src/gestor_citas.py	Funciones puras e inmutabilidad conservadas de la entrega funcional.
src/mvp.py	Modelos y operaciones de agendas, pacientes y médicos con transacciones SQLite.
src/autenticacion.py	Cinco cuentas de demostración, PBKDF2-SHA256, tokens temporales y autorización.
src/servidor.py	Servidor HTTP y endpoints protegidos por sesión y rol.
frontend/mvp.html, mvp.css, mvp.js	Acceso inicial y paneles específicos según el perfil.
tests/	Pruebas funcionales y de permisos de las rutas HTTP.

11.1 Endpoints del servidor

Método y ruta	Acceso / resultado
POST /api/mvp/auth/login	Todos; devuelve token en caso válido
GET /api/mvp/auth/me; POST /api/mvp/auth/logout	Sesión actual / terminar sesión
GET /api/mvp/doctores y /api/mvp/disponibilidad	Cuentas autenticadas; doctor limitado a su médico
GET /api/mvp/citas	Solo perfiles autorizados; agendas filtradas
POST /api/mvp/citas y /citas/{id}/cancelar	Administrador y recepción
GET /api/mvp/resumen y /api/mvp/usuarios	Solo administrador
POST /api/mvp/doctores	Solo administrador

11.2 Consideraciones técnicas

La validación de empalmes ocurre dentro de una transacción SQLite, lo que impide confirmar dos citas simultáneas para el mismo médico y horario. Se revisan fecha, duración, identificadores y campos del paciente. Las contraseñas se derivan mediante PBKDF2-HMAC-SHA256 con sal; los tokens aleatorios permanecen en memoria y se invalidan al cerrar sesión o reiniciar el servidor.

12. Pruebas y alcance de la evaluación

Se ejecutaron 53 pruebas automatizadas locales con unittest, todas aprobadas. Se incluyen las 33 pruebas de negocio existentes y verificaciones nuevas de autenticación, permisos, privacidad por rol, registro de médicos y funcionamiento HTTP. La ejecución visual completa del navegador queda pendiente de comprobar en el equipo del laboratorio.

Caso para laboratorio	Resultado esperado
Entrar con contraseña incorrecta	Se rechaza el acceso, sin abrir el panel.
Entrar con cualquiera de las cinco cuentas	Se abre el panel correspondiente al rol.
Recepción registra cita en horario libre	Aparece en agenda y ocupa el intervalo seleccionado.
Intentar reservar horario que se empalma	Se muestra error; no se crea cita duplicada.
Cancelar cita y consultar horarios	La cita cambia a cancelada y el intervalo queda disponible.
Doctor intenta consultar otro médico	Servidor responde con acceso denegado (403).
Paciente consulta sus citas	Solo muestra las vinculadas a su usuario.
Personal autorizado intenta listar citas	El servidor rechaza la operación (403).
Administrador crea médico nuevo	Aparece en catálogo; ID duplicado es rechazado.

12.1 Límites declarados para la evaluación

No se ofrecen como parte del demo: reprogramación, notificaciones, edición o eliminación de usuarios, expediente médico, aviso de inasistencia del doctor, envío de correos, despliegue remoto o seguridad apta para información de salud real. El usuario “personal autorizado” tiene solamente permisos de consulta pública de médicos y espacios.

12.2 Importancia del alcance

La profesora elegirá tres pruebas aleatorias a partir de lo descrito en el reporte. Por ello las funcionalidades anteriores se expresan como hechos comprobados con pruebas automáticas; los requisitos aún no desarrollados se mantienen identificados como trabajo posterior.

13. Ejecución del demo y conclusiones

13.1 Preparar el laboratorio

- Abrir la carpeta completa del repositorio en Visual Studio Code.
- Ejecutar python app.py, o INICIAR.bat desde Windows.
- Entrar a la dirección local que muestre Python (normalmente http://127.0.0.1:8000).
- Seleccionar una de las cinco cuentas en la pantalla de inicio de sesión. La contraseña común es Demo2026!.
- Para comprobar el panel de paciente, iniciar primero como recepción y agendar una cita marcando “Vincular cita con la cuenta paciente”. Después cerrar sesión e ingresar como paciente.
- Para revisar los permisos, intentar una operación no autorizada con otro rol. Las respuestas 401/403 se generan en Python.
- Para lanzar todas las pruebas, ejecutar python -m unittest discover -s tests -v.

13.2 Recomendaciones de demostración

Es preferible utilizar fechas de práctica y datos inventados. El sistema guarda el estado en data/hospital.sqlite3: los registros persisten hasta que se elimine intencionalmente esa base de prueba. Para obtener una base limpia, cerrar Python y borrar únicamente ese archivo; el programa lo reconstruirá.

13.3 Conclusiones

La implementación amplía la búsqueda funcional de horarios del primer parcial con registros persistentes, acceso por cinco perfiles de demostración y verificación de permisos en el servidor. La separación entre autenticación, lógica de negocio y presentación permite agregar requisitos futuros sin mezclar responsabilidades. El MVP se limita a funciones verificables y utiliza exclusivamente datos ficticios.

Repositorio de trabajo (rama de segundo parcial):
https://github.com/luisgabrielrosas4-glitch/Gestor_Hospital/tree/segundo-parcial-mvp-2026

Integrantes: Contreras Cruz Joseph · Mendez Gamboa Santiago · Rosas Rios Luis Gabriel.